SmartClass

Administrator and Operations Manual

Version 1.0 | 27 October 2026

For the school administrator and the person who runs the server

Screenshots come from a demonstration school with invented data. Secrets in this manual are placeholders (`CHANGE_ME`).

Contents

1 Roles and responsibilities **2**

2 Architecture in one page **2**

3 First steps after installation **2**

4 People **3**

4.1 Approving students 3

4.2 Teachers 4

5 School settings **4**

5.1 Artificial intelligence 4

5.2 WhatsApp 5

5.3 Privacy and safeguarding 6

6 Backups and recovery **6**

6.1 From the application 6

6.2 From the command line 7

6.3 Recovery procedure 7

7 Monitoring and routine maintenance **8**

8 Security operations **8**

9 Troubleshooting reference **9**

10 Quality evidence **9**

1 Roles and responsibilities

Role	Responsible for
Super admin (school administrator)	approving students and teachers, school settings, privacy, AI and WhatsApp rules, grading scales, backups from the application, reading the system check
Server operator	installation, upgrades, cron jobs, database and file backups, web-server and PHP settings, TLS, monitoring (see the Installation Guide)
Teacher	only their own classes

2 Architecture in one page

- **Code:** plain PHP 8 (no framework). `pages/<module>/` one file per screen; `includes/` shared code (not web-reachable); `config/` database connection, `install_schema.sql` (fresh install), `migrations/` (changes to an existing database); `bin/` command-line tools; `cron/` scheduled jobs; `tests/` automated tests.
- **One request:** the page loads `config/db.php`, which finds the school (by domain), opens that school's database, starts the session and sets security headers; `includes/auth.php` stops visitors who are not signed in and students who are not approved; the page checks who may do the action, checks the CSRF token token on every POST, uses prepared statements, and redirects.
- **Data:** MariaDB/MySQL (76 tables, 62 migrations in this version); uploaded files in `uploads/` under random names; backups in `backups/`; sessions in `sessions/`; the PHP error log in `logs/php-error.log` (pages never print errors).
- **Several schools:** each customer has `config/instances/<domain>.php` (database, storage folder) and its own database; one code folder serves all. See `docs/CUSTOMERS.md`.
- **Languages:** English text is written in the pages; Pashto and Dari are produced by a translation layer from dictionaries in `includes/lang/`.

3 First steps after installation

1. Sign in with the account created by the installer. Change the password if it is temporary (**Settings** → **Password & security**, figure 1); set up two-step sign-in (**Two-step sign-in**).
2. Open **Settings** → **System** → **System check**: it reports folder permissions and when the background jobs last ran. Fix every warning.
3. Fill in the school settings (name, logo, languages, grading scale) and the rules and policies.
4. Switch e-mail on (**Settings** → **Communication** → **E-mail**) so password reset and notifications can be sent.

5. Invite teachers; approve students (section 4.1).
6. Take and *test* a backup (section 6).

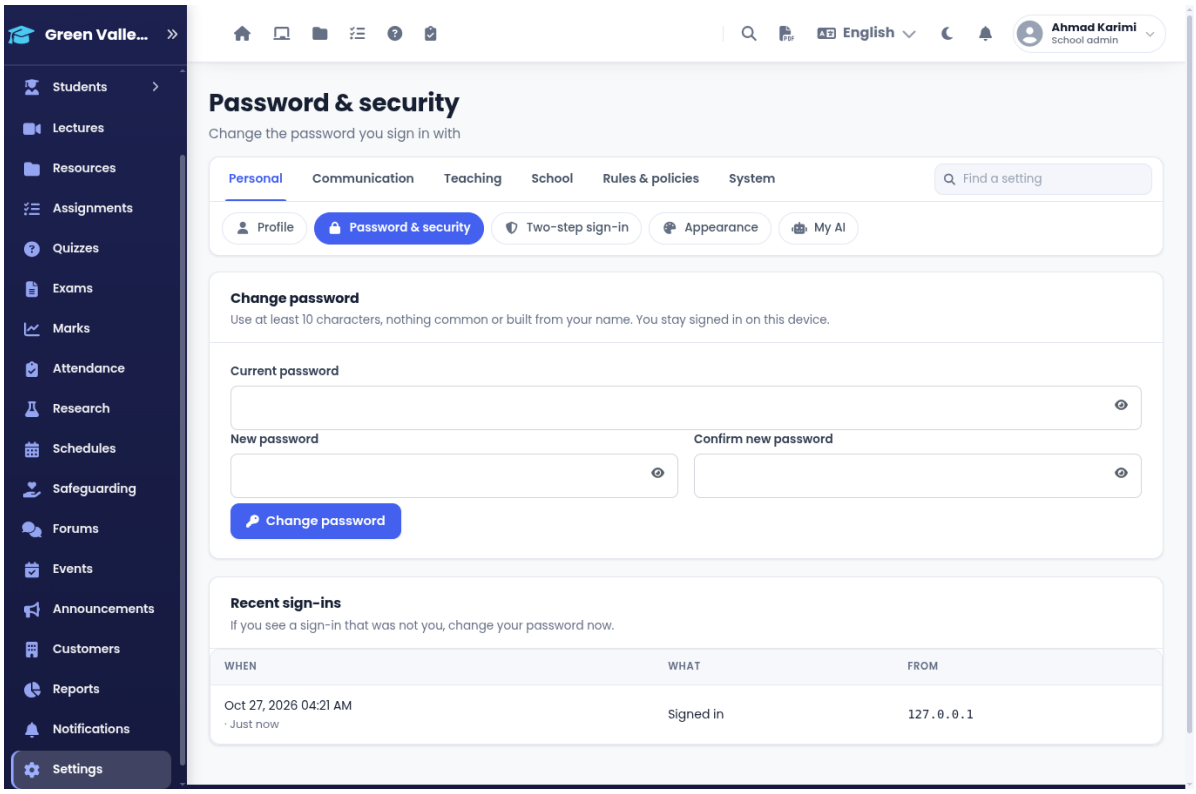


Figure 1: Personal settings: password and recent sign-ins.

4 People

4.1 Approving students

New students wait until approved (figure 2). **Review** shows the account, **Approve** lets them in, **Reject** refuses (a confirmation is asked). Teachers can also approve join requests for their own classes.

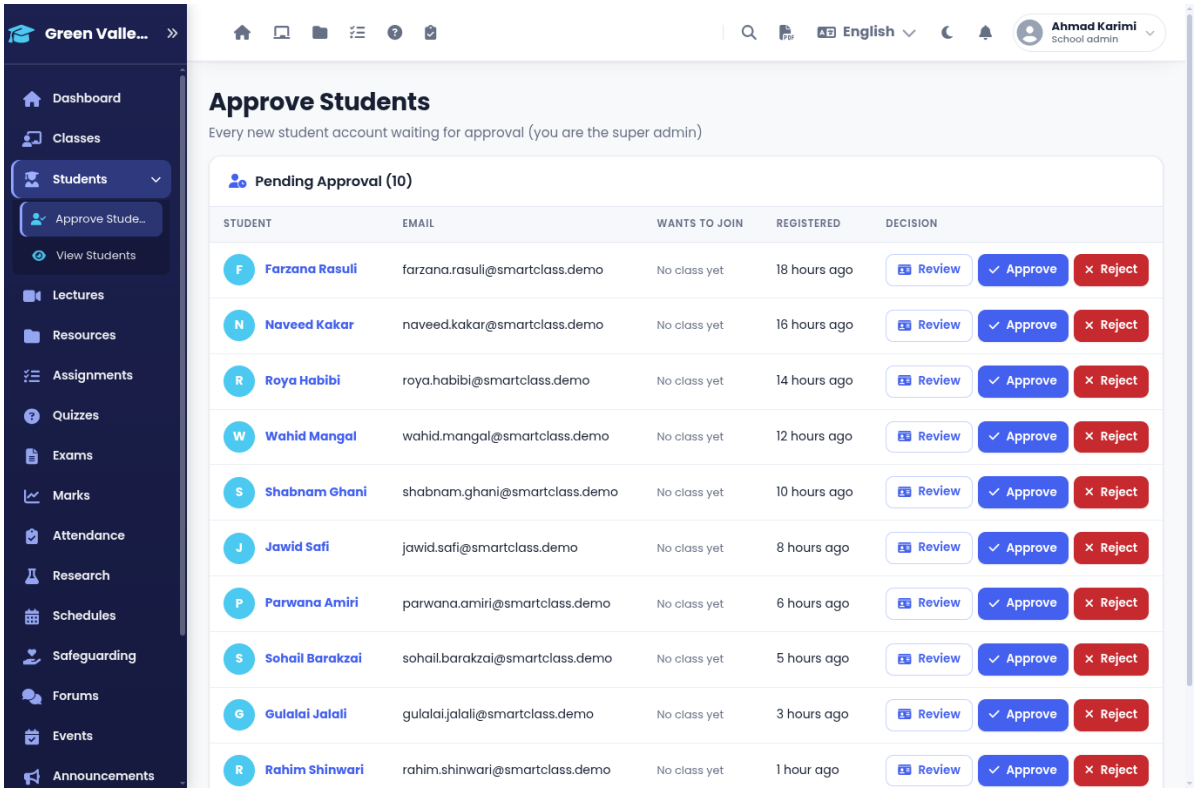


Figure 2: Students waiting for approval.

4.2 Teachers

Teachers join through an invitation from the administrator (the invitation page is `pages/auth/teacher_invite.php`). A teacher sees only their own classes. The first teacher created at installation is the super admin.

5 School settings

All settings are under **Settings**, in six groups: **Personal** (profile, password, two-step sign-in, appearance, my AI), **Communication** (notifications, WhatsApp, WhatsApp numbers, e-mail), **Teaching**, **School**, **Rules & policies** and **System** (AI providers, code runner, system check, backup). The search box “Find a setting” jumps to a setting by name. Nothing the administrator may want to change is hard-coded.

5.1 Artificial intelligence

System → **AI providers** (figure 3) holds the master switch: when off, teachers and students see no AI button anywhere and every feature still works by hand. When on, choose a default provider (and a fallback), optionally a different provider per feature (quiz questions, grading help, class insights, …) and keep an eye on the monthly usage. AI only drafts: nothing is published, graded or e-mailed without a teacher. Provider keys are stored encrypted and never shown again. Teachers decide per class whether students get the AI writing help.

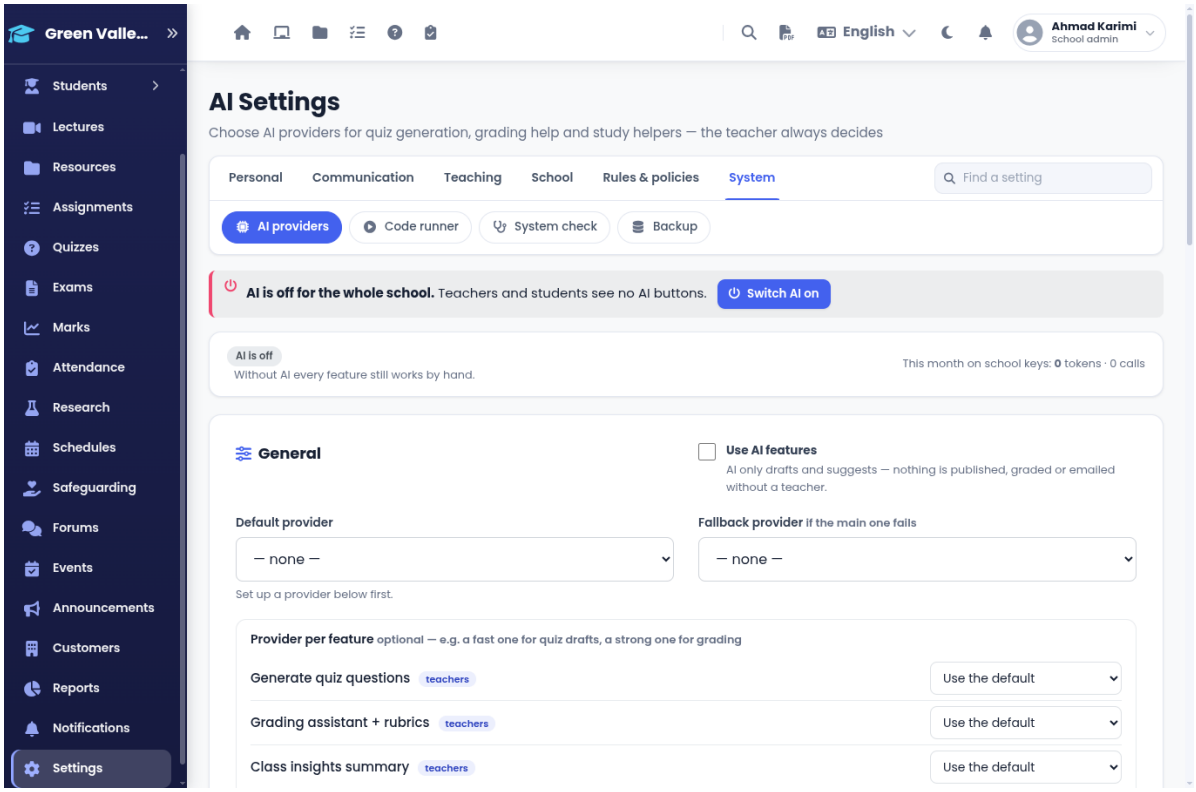


Figure 3: AI settings (here: switched off for the school).

5.2 WhatsApp

Communication → **WhatsApp** (figure 4). WhatsApp is optional. **Level 1** is a button that opens WhatsApp with the message ready; the person presses Send. **Level 2** sends by itself through the official WhatsApp Business service from a sender set up on this page. The page also holds your own number and consent, the school rules (the switch for the whole school), message texts per language and the log. A person receives messages only after agreeing, and can stop at any time. Details: `docs/WHATSAPP.md`. Real delivery depends on your own Meta business account and was not verified in the test environment.

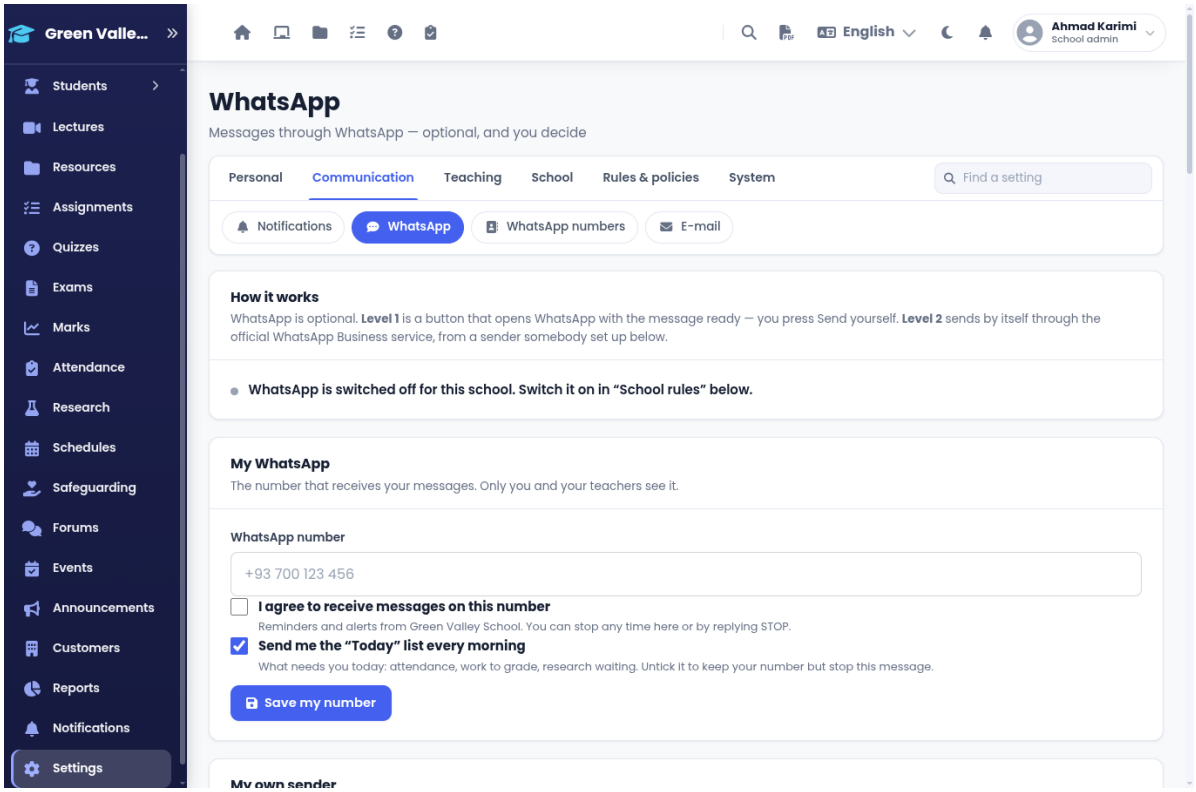


Figure 4: WhatsApp settings (switched off for the school).

5.3 Privacy and safeguarding

Rules & policies contains the privacy rules (retention, export, deletion) and safeguarding. `docs/PRIVACY.md` states who can read what; `docs/AI_POLICY_TEMPLATE.md` is a template for the school's AI rules, which should be agreed with parents and teachers before AI is switched on for students.

6 Backups and recovery

6.1 From the application

System → **Backup** (figure 5) downloads a complete copy of the database as one compressed file. Uploaded files are *not* in it. The file contains accounts, password hashes and saved keys – keep it private. **Restore a backup** replaces everything with the backup (a safety copy of the current data is saved first) and asks for your password and the word `RESTORE`.

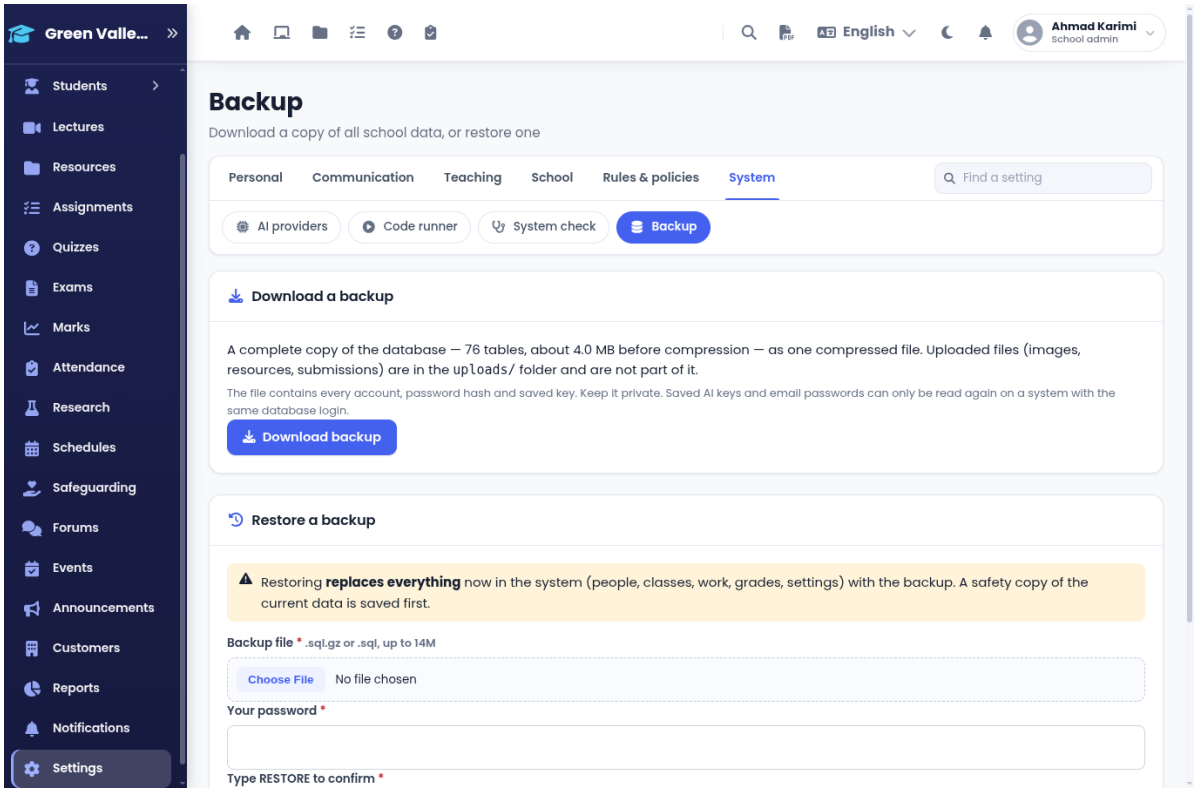


Figure 5: Download and restore a backup.

6.2 From the command line

```
php bin/backup_all.php                    # every school -> backups/all/<date>/ (14
    days kept)
php bin/backup_offsite.php --to=user@host:/backups/smartclass
php bin/restore_drill.php                 # proves a backup can be restored (read-only
    on live data)
```

Schedule the first in cron (Installation Guide) and copy backups *off the server* – a backup on the same disk is not protection against loss of the server. Copy `uploads/` with your normal file backup.

6.3 Recovery procedure

1. Put the maintenance page on / stop the web server.
2. Restore the code from Git or the installation package; put back `config/local.php` (with the *same* `secret_key`) and `config/instances/`.
3. Create an empty database and import the newest good backup: `gunzip -c file.sql.gz | mysql -u USER -p DB`.
4. Restore `uploads/`.
5. Run `php bin/migrate.php` (applies anything newer than the backup), start the web server, open **System check**.
6. Tell users what time period of data may be missing (everything after the backup).

Run `php bin/restore_drill.php` regularly: it restores a fresh backup into a throw-away database and compares every table (row counts, checksums) and all foreign keys. On the test system it reported 76 tables identical in about 2 seconds.

7 Monitoring and routine maintenance

When	What
Daily (automatic)	`cron/housekeeping.php` records that it ran, archives the error log when it exceeds 2 MB and tells administrators if something went wrong in the last 24 hours or a job is late. `cron/send_emails.php` (every 5 minutes) sends e-mail and WhatsApp messages and publishes scheduled posts.
Weekly	open **System check**; read the tail of `logs/php-error.log`; check free disk space for `uploads/` and `backups/`; confirm that the off-server backup arrived.
Monthly	run the restore drill; review users (disable leavers); check AI usage against the budget.
After any change	`sh bin/check.sh` on a copy (the full test suite, about 2.5 minutes) before it goes live.

Capacity. A normal page takes a fraction of a second; a streamed AI answer keeps one PHP worker busy for its whole duration (5–30 s). Plan workers as in `docs/CAPACITY.md`; measure with the FPM status page during a real lesson. On the test machine (single worker, PHP built-in server) the key pages answered in 2–30 ms with 120 students and about 1 400 submissions; this is not a promise for your server.

8 Security operations

- **Secrets:** `config/local.php` (database login, `secret_key`) is never committed. Rotate the database password if it was ever written in code or shared, then update `local.php`; run `php bin/rotate_secrets.php` to tie stored secrets to the independent key.
- **Web exposure:** after every deployment check that `.git/config`, `config/local.php`, `logs/php-error.log`, `tests/` and `backups/` are *not* downloadable (commands in the Installation Guide). Delete `setup.php`.
- **HTTPS:** serve only over HTTPS; the application can force it (**Rules & policies**).
- **Sign-in protection:** 8 wrong passwords lock an account for a while; a school can require two-step sign-in for administrators; reset a lost two-step device with `php bin/reset_two_factor.php`.
- **Updates:** keep PHP, MariaDB and the operating system patched.
- **Known open items** of this version are listed in `qa/release-readiness.md` (for example the missing site-wide Content-Security-Policy).

9 Troubleshooting reference

Symptom	Cause and action
Everyone gets a database error	`config/local.php` missing/wrong, or MariaDB stopped: `systemctl status mariadb`; test with `mysql -u USER -p DB`.
Blank page, HTTP 500	read `logs/php-error.log`; the page never prints errors.
Uploads fail	folder rights (**System check**); PHP limits (`upload_max_filesize 14M`, `post_max_size 20M`); disk full.
No e-mail / WhatsApp	cron line for `send_emails.php` missing; e-mail or WhatsApp not configured; the sender's token expired (WhatsApp page log).
System check says cron did not run	the crontab is for the wrong user; run the command by hand as the web-server user and read the output.
Migration fails	the command prints file and SQL error; fix and rerun; finished files are not repeated. Restore the pre-update backup if needed.
A hosted school shows "paused" or "unknown site"	Customers page (status), `config/instances/<domain>.php`, DNS/virtual host for the domain.
Administrator locked out	use another administrator; or reset the password with `php cron/reset_password.php`; two-step device lost: `php bin/reset_two_factor.php`.
AI answers do not appear live	reverse proxy buffering: `proxy_buffering off;`.
Pages unstyled	the files in `assets/vendor/` are missing or blocked (check the exposure rules do not deny `assets/`); reload with the cache cleared.

10 Quality evidence

The test suite (`php tests/run.php`) runs on a throw-away database and drives the real pages: 169 tests passed in the final run of this release, including a route matrix of 2 330 requests, 1 152 injection and script probes, upload attacks, accessibility checks and a performance check. Reports are in `qa/`.

Glossary

CSRF token A hidden value in every form that proves the request came from the page the person was looking at.. 2

Super admin A teacher account with the administrator flag. Approves users and owns the school settings, privacy, AI and WhatsApp rules and backups.. 2